

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ
по лабораторной работе № 5
по курсу «Разработка кроссплатформенных приложений»
Вариант 3

Выполнили
студенты группы 22ВОЭ1
Хворов Ф. М.
Адов А. А.

Приняли
Юрова О. В.

Пенза 2025

Цель работы

Научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

Задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках (число потоков определяется номером варианта), снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса.

Исходный код программы

```
import javax.swing.JOptionPane;
import javax.swing.*;
import java.io.*;
import javax.swing.table.DefaultTableModel;
import java.util.ArrayList;

public class NewJFrame extends javax.swing.JFrame {

    private ArrayList<RecIntegral> integralsList;
    public NewJFrame() {
        initComponents();
        integralsList = new ArrayList<>();
    }

    public void Table(){

    }

    @SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jButton1 = new javax.swing.JButton();
    jScrollPane1 = new javax.swing.JScrollPane();
    jTable1 = new javax.swing.JTable();
    jButton2 = new javax.swing.JButton();
    jButton3 = new javax.swing.JButton();
    jTextFieldLowLim = new javax.swing.JTextField();
    jTextFieldUpLim = new javax.swing.JTextField();
    jTextFieldStep = new javax.swing.JTextField();
    jLabel1 = new javax.swing.JLabel();
    jLabel2 = new javax.swing.JLabel();
    jLabel3 = new javax.swing.JLabel();
    jButton4 = new javax.swing.JButton();
    jButton5 = new javax.swing.JButton();
    jLabel4 = new javax.swing.JLabel();
    jLabel5 = new javax.swing.JLabel();
    jLabel6 = new javax.swing.JLabel();
    jButton6 = new javax.swing.JButton();
    jButton7 = new javax.swing.JButton();
    jButton8 = new javax.swing.JButton();
    jButton9 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    jButton1.setText("Удалить");
    jButton1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton1ActionPerformed(evt);
        }
    });

    jTable1.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {

        },
        new String [] {
            "Нижняя граница", "Верхняя граница", "Шаг", "Результат"
        }
    ) {
        boolean[] canEdit = new boolean [] {
            false, false, false, false
        };
    });
}
// </editor-fold>

```

```

    };

    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
});
jScrollPane1.setViewportView(jTable1);

jButton2.setText("Рассчитать");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});

jButton3.setText("Добавить");
jButton3.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton3ActionPerformed(evt);
    }
});

jTextFieldLowLim.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jTextFieldLowLimActionPerformed(evt);
    }
});

jLabel1.setText("Нижняя граница");

jLabel2.setText("Верхняя граница");

jLabel3.setText("Шаг интегрирования");

jButton4.setText("Заполнить");
jButton4.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton4ActionPerformed(evt);
    }
});

jButton5.setText("Очистить");
jButton5.addActionListener(new java.awt.event.ActionListener() {

```

```
        public void actionPerformed(java.awt.event.ActionEvent evt) {  
            jButton5ActionPerformed(evt);  
        }  
    });
```

```
jLabel4.setText("Работа со списком");
```

```
jLabel5.setText("Работа с текстом");
```

```
jLabel6.setText("Работа с двоичным кодом");
```

```
jButton6.setText("Загрузка");  
jButton6.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        jButton6ActionPerformed(evt);  
    }  
});
```

```
jButton7.setText("Сохранение");  
jButton7.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        jButton7ActionPerformed(evt);  
    }  
});
```

```
jButton8.setText("Загрузка");  
jButton8.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        jButton8ActionPerformed(evt);  
    }  
});
```

```
jButton9.setText("Сохранение");  
jButton9.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        jButton9ActionPerformed(evt);  
    }  
});
```

```
javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());  
getContentPane().setLayout(layout);  
layout.setHorizontalGroup(  
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```

        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup())
        .addGap(28, 28, 28)
            .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 441,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(44, 44, 44)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
            .addGroup(javax.swing.GroupLayout.Alignment.LEADING, layout.createSequentialGroup())
            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
                .addComponent(jButton2)
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(jButton1)
                    .addComponent(jButton3)))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup())
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
                    .addComponent(jButton4)
                    .addComponent(jButton5))
                .addGap(8, 8, 8))
            .addComponent(jLabel4, javax.swing.GroupLayout.Alignment.TRAILING))
        .addGap(24, 24, 24))
        .addGroup(javax.swing.GroupLayout.Alignment.LEADING, layout.createSequentialGroup())
        .addComponent(jLabel5)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 86,
Short.MAX_VALUE)
        .addComponent(jLabel6)
        .addContainerGap()
        .addGroup(layout.createSequentialGroup())
        .addComponent(jButton6)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton8)
        .addGap(46, 46, 46))
        .addGroup(layout.createSequentialGroup())
        .addComponent(jButton7)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton9)
        .addGap(36, 36, 36)))
        .addGroup(layout.createSequentialGroup())
        .addGap(41, 41, 41)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jLabel1)

```

```

        .addComponent(jTextFieldLowLim, javax.swing.GroupLayout.PREFERRED_SIZE, 107,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(40, 40, 40)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jLabel2)
            .addComponent(jTextFieldUpLim, javax.swing.GroupLayout.PREFERRED_SIZE, 118,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(35, 35, 35)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
            .addComponent(jLabel3, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addComponent(jTextFieldStep))
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup()
            .addContainerGap(32, Short.MAX_VALUE)
            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(jLabel1)
                .addComponent(jLabel2)
                .addComponent(jLabel3))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(jTextFieldLowLim, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addComponent(jTextFieldUpLim, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addComponent(jTextFieldStep, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
            .addGap(62, 62, 62)
            .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 288,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(14, 14, 14))
        .addGroup(layout.createSequentialGroup()
            .addGroup(layout.createSequentialGroup()
                .addGap(92, 92, 92)
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addGroup(layout.createSequentialGroup()
                        .addComponent(jLabel4)
                        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                        .addComponent(jButton4)
                        .addGap(18, 18, 18)
                        .addComponent(jButton5))
                    .addGroup(layout.createSequentialGroup()
                        .addGroup(layout.createSequentialGroup()
                            .addComponent(jButton3)

```

```

        .addGap(18, 18, 18)
        .addComponent(jButton1)
        .addGap(18, 18, 18)
        .addComponent(jButton2)))
    .addGap(38, 38, 38)
    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jLabel5)
        .addComponent(jLabel6))
    .addGap(18, 18, 18)
    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jButton6)
        .addComponent(jButton8))
    .addGap(18, 18, 18)
    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jButton7)
        .addComponent(jButton9))
    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
);

pack();
} // </editor-fold>

```

```

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();

    int rowNum = jTable1.getSelectedRow();

    if (rowNum == -1) {
        JOptionPane.showMessageDialog(null, "выберите строку");
    } else tModel.removeRow(rowNum);
}

private void jTextFieldLowLimActionPerformed(java.awt.event.ActionEvent evt) {

}

private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {

    try {
        double upperLim;
        double lowLim;
        double step;

```

```

// Парсим введенные данные
lowLim = Double.parseDouble(jTextFieldLowLim.getText());
upperLim = Double.parseDouble(jTextFieldUpLim.getText());
step = Double.parseDouble(jTextFieldStep.getText());

// Создаем экземпляр RecIntegral с проверкой данных
RecIntegral recIntegral = new RecIntegral(lowLim, upperLim, step, 0.0);

// Добавляем строку в таблицу
DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
tModel.addRow(new Object[]{lowLim, upperLim, step});

// Добавляем запись в коллекцию
integralsList.add(recIntegral);

// Очищаем поля ввода
jTextFieldLowLim.setText("");
jTextFieldUpLim.setText("");
jTextFieldStep.setText("");
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Введены некорректные данные. Пожалуйста, введите
числа.", "Ошибка", JOptionPane.ERROR_MESSAGE);
} catch (InvalidDataException e) {
    JOptionPane.showMessageDialog(this, e.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
}

}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    long startTime = System.nanoTime();
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();

    for (int row = 0; row < tModel.getRowCount(); row++) {
        final int finalRow = row; // Финализируем переменную для лямбды

        double a = ((Number) tModel.getValueAt(finalRow, 0)).doubleValue();
        double b = ((Number) tModel.getValueAt(finalRow, 1)).doubleValue();
        double step = ((Number) tModel.getValueAt(finalRow, 2)).doubleValue();

        // Разбиение интервала на 3 потока
        double interval = (b - a) / 3;

        IntegralCalculatorThread thread1 = new IntegralCalculatorThread(a, a + interval, step);

```

```

IntegralCalculatorThread thread2 = new IntegralCalculatorThread(a + interval, a + 2 * interval, step);
IntegralCalculatorThread thread3 = new IntegralCalculatorThread(a + 2 * interval, b, step);

thread1.start();
thread2.start();
thread3.start();

try {
    thread1.join();
    thread2.join();
    thread3.join();

    double totalIntegral = thread1.getPartialSum() + thread2.getPartialSum() + thread3.getPartialSum();

    SwingUtilities.invokeLater(() -> {
        tModel.setValueAt(totalIntegral, finalRow, 3);
        integralsList.get(finalRow).setResult(totalIntegral);
    });

} catch (InterruptedException e) {
    JOptionPane.showMessageDialog(this, "Ошибка: " + e.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
}

JOptionPane.showMessageDialog(this, "Вычисления завершены!", "Готово",
JOptionPane.INFORMATION_MESSAGE);

long endTime = System.nanoTime();
long executionTime = endTime - startTime;
System.out.println("Vremya vipolneniya: " + executionTime / 1_000_000 + " ms");

}

class IntegralCalculatorThread extends Thread {
private final double a;    // Начало интервала
private final double b;    // Конец интервала
private final double step; // Шаг интегрирования
private double partialSum; // Частичная сумма (результат потока)

public IntegralCalculatorThread(double a, double b, double step) {
    this.a = a;

```

```

        this.b = b;
        this.step = step;
        this.partialSum = 0;
    }

    @Override
    public void run() {
        double sum = 0;
        for (double x = a; x < b; x += step) {
            sum += Math.cos(x + step / 2) * step; // Метод средних прямоугольников
        }
        this.partialSum = sum;
    }

    public double getPartialSum() {
        return partialSum;
    }
}

```

```

private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
    tModel.setRowCount(0);

```

```

        for (RecIntegral rec : integralsList) {
            tModel.addRow(new Object[] {
                rec.getLowerLimit(),
                rec.getUpperLimit(),
                rec.getStep(),
                rec.getResult()
            });
        }
    }
}

```

```

private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {
    DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
    tModel.setRowCount(0);
    integralsList.clear();
}

```

```

private void jButton7ActionPerformed(java.awt.event.ActionEvent evt) {
    JFileChooser fileChooser = new JFileChooser();

```

```

        if (fileChooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
            File file = fileChooser.getSelectedFile();
            saveToTextFile(file);
        }
    }

    private void jButton8ActionPerformed(java.awt.event.ActionEvent evt) {
        JFileChooser fileChooser = new JFileChooser();
        if (fileChooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
            File file = fileChooser.getSelectedFile();
            loadFromBinaryFile(file);
        }
    }

    private void jButton9ActionPerformed(java.awt.event.ActionEvent evt) {
        JFileChooser fileChooser = new JFileChooser();
        if (fileChooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
            File file = fileChooser.getSelectedFile();
            saveToBinaryFile(file);
        }
    }

    private void jButton6ActionPerformed(java.awt.event.ActionEvent evt) {
        JFileChooser fileChooser = new JFileChooser();
        if (fileChooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
            File file = fileChooser.getSelectedFile();
            loadFromTextFile(file);
        }
    }

    public class InvalidDataException extends Exception {
        public InvalidDataException(String message) {
            super(message);
        }
    }

    private void saveToTextFile(File file) {
        try (PrintWriter writer = new PrintWriter(file)) {
            for (RecIntegral rec : integralsList) {
                writer.println(rec.getLowerLimit() + "," + rec.getUpperLimit() + "," + rec.getStep() + "," +
rec.getResult());
            }
            JOptionPane.showMessageDialog(this, "Данные успешно сохранены в текстовый файл.", "Успех",
JOptionPane.INFORMATION_MESSAGE);
        } catch (IOException e) {

```

```

        JOptionPane.showMessageDialog(this, "Ошибка при сохранении файла: " + e.getMessage(),
"Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}

private void loadFromTextFile(File file) {
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        integralsList.clear();
        DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
        tModel.setRowCount(0);
        String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split(",");
            double lowerLimit = Double.parseDouble(parts[0]);
            double upperLimit = Double.parseDouble(parts[1]);
            double step = Double.parseDouble(parts[2]);
            double result = Double.parseDouble(parts[3]);
            RecIntegral rec = new RecIntegral(lowerLimit, upperLimit, step, result);
            integralsList.add(rec);
            tModel.addRow(new Object[] {lowerLimit, upperLimit, step, result});
        }

        JOptionPane.showMessageDialog(this, "Данные успешно загружены из текстового файла.",
"Успех", JOptionPane.INFORMATION_MESSAGE);
    } catch (IOException | InvalidDataException e) {
        JOptionPane.showMessageDialog(this, "Ошибка при загрузке файла: " + e.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
    }
}

private void saveToBinaryFile(File file) {
    System.out.println("Сохранение данных: " + integralsList);
    if (integralsList == null || integralsList.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Нет данных для сохранения.", "Ошибка",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    try (FileOutputStream fileOut = new FileOutputStream(file)) {
        ObjectOutputStream out = new ObjectOutputStream(fileOut);
        out.writeObject(integralsList);
        JOptionPane.showMessageDialog(this, "Данные успешно сохранены в двоичный файл.", "Успех",
JOptionPane.INFORMATION_MESSAGE);
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "Ошибка при сохранении файла: " + e.getMessage(),
"Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}

```

```

    } catch (Exception ex) {
        throw new RuntimeException(ex);
    }

    //integralsList.removeAll(integralsList);

}

private void loadFromBinaryFile(File file) {
    if (!file.exists() || file.length() == 0) {
        JOptionPane.showMessageDialog(this, "Файл не существует или пуст.", "Ошибка",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) {
        integralsList = (ArrayList<RecIntegral>) ois.readObject(); // Десериализуем только integralsList

        // Обновляем таблицу
        DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();
        tModel.setRowCount(0); // Очищаем таблицу
        for (RecIntegral rec : integralsList) {
            tModel.addRow(new Object[]{
                rec.getLowerLimit(),
                rec.getUpperLimit(),
                rec.getStep(),
                rec.getResult()
            });
        }

        JOptionPane.showMessageDialog(this, "Данные успешно загружены из двоичного файла.", "Успех",
JOptionPane.INFORMATION_MESSAGE);
    } catch (IOException | ClassNotFoundException e) {
        JOptionPane.showMessageDialog(this, "Ошибка при загрузке файла: " + e.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
    }
}

public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new NewJFrame().setVisible(true);
        }
    });
}

```

```
}

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JButton jButton3;
private javax.swing.JButton jButton4;
private javax.swing.JButton jButton5;
private javax.swing.JButton jButton6;
private javax.swing.JButton jButton7;
private javax.swing.JButton jButton8;
private javax.swing.JButton jButton9;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel5;
private javax.swing.JLabel jLabel6;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable jTable1;
private javax.swing.JTextField jTextFieldLowLim;
private javax.swing.JTextField jTextFieldStep;
private javax.swing.JTextField jTextFieldUpLim;
// End of variables declaration
}
```

Выполнение программы

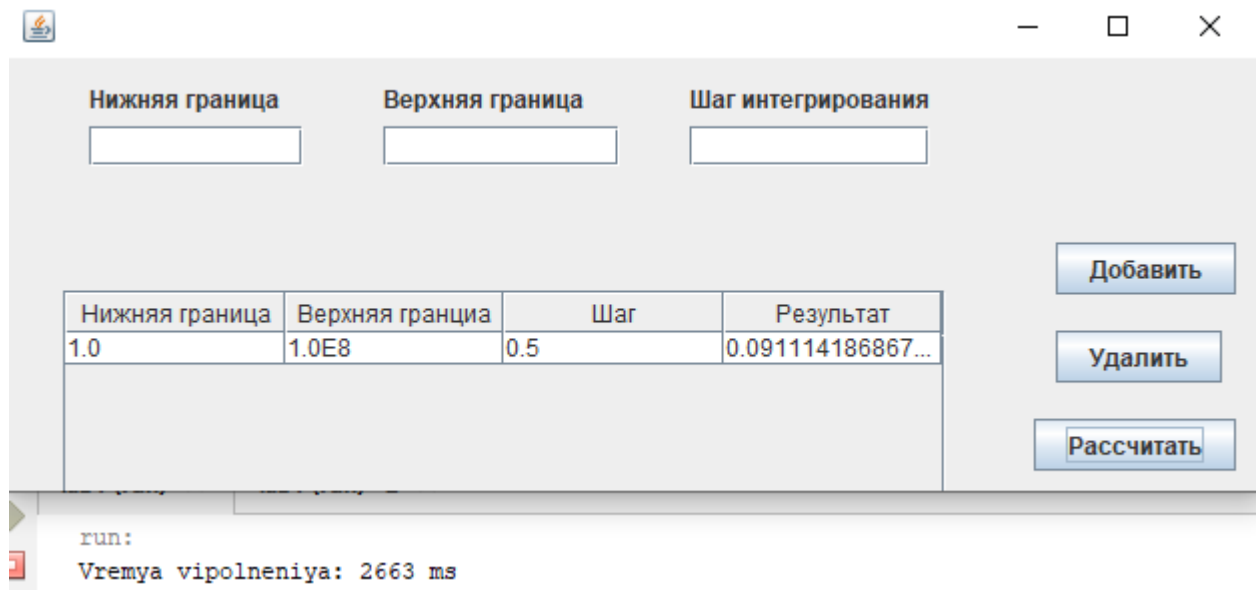


Рисунок 1 — Время работы с потоками

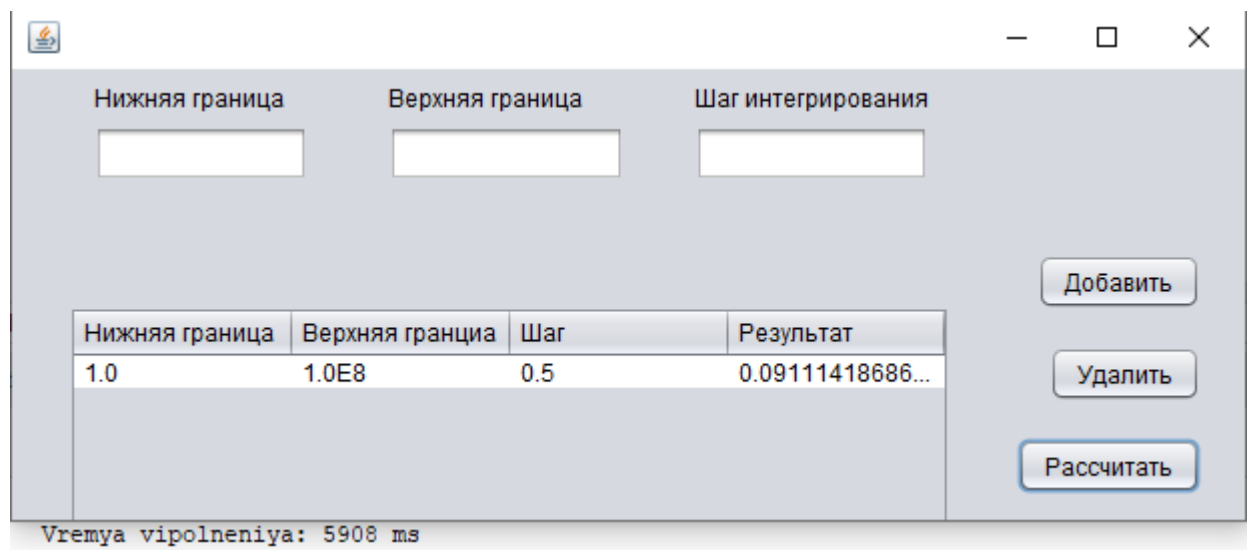


Рисунок 2 — Время работы без потоков

Ход работы

```
long startTime = System.nanoTime();

DefaultTableModel tModel = (DefaultTableModel) jTable1.getModel();

for (int row = 0; row < tModel.getRowCount(); row++) {
    final int finalRow = row; // Финализируем переменную для лямбды

    double a = ((Number) tModel.getValueAt(finalRow, 0)).doubleValue();
    double b = ((Number) tModel.getValueAt(finalRow, 1)).doubleValue();
    double step = ((Number) tModel.getValueAt(finalRow, 2)).doubleValue();

    // Разбиение интервала на 3 потока
    double interval = (b - a) / 3;

    IntegralCalculatorThread thread1 = new IntegralCalculatorThread(a, a + interval, step);
    IntegralCalculatorThread thread2 = new IntegralCalculatorThread(a + interval, a + 2 * interval, step);
    IntegralCalculatorThread thread3 = new IntegralCalculatorThread(a + 2 * interval, b, step);

    thread1.start();
    thread2.start();
    thread3.start();

    try {
        thread1.join();
        thread2.join();
        thread3.join();

        double totalIntegral = thread1.getPartialSum() + thread2.getPartialSum() + thread3.getPartialSum();

        SwingUtilities.invokeLater(() -> {
            tModel.setValueAt(totalIntegral, finalRow, 3);
            integralsList.get(finalRow).setResult(totalIntegral);
        });

    } catch (InterruptedException e) {
        JOptionPane.showMessageDialog(this, "Ошибка: " + e.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);
    }
}
```

```
JOptionPane.showMessageDialog(this, "Вычисления завершены!", "Готово",  
JOptionPane.INFORMATION_MESSAGE);
```

```
        long endTime = System.nanoTime();  
        long executionTime = endTime - startTime;  
        System.out.println("Время выполнения: " + executionTime / 1_000_000 + " ms");  
    }  
    class IntegralCalculatorThread extends Thread {  
        private final double a;    // Начало интервала  
        private final double b;    // Конец интервала  
        private final double step; // Шаг интегрирования  
        private double partialSum; // Частичная сумма (результат потока)  
  
        public IntegralCalculatorThread(double a, double b, double step) {  
            this.a = a;  
            this.b = b;  
            this.step = step;  
            this.partialSum = 0;  
        }  
  
        @Override  
        public void run() {  
            double sum = 0;  
            for (double x = a; x < b; x += step) {  
                sum += Math.cos(x + step / 2) * step; // Метод средних прямоугольников  
            }  
            this.partialSum = sum;  
        }  
  
        public double getPartialSum() {  
            return partialSum;  
        }  
    }  
}
```

Пояснение к тексту программы(основные вычисления)

Строки 3 — 8: Финализируем переменную для лямбды.

Строки 10 — 14: Разбиение интервала на 3 потока.

Строки 16 — 23: Запуск работы 3 потоков и приостановление выполнение текущего потока до тех пор, пока поток, к которому применён метод `join`, не закончит свою работу.

Строка 24: Суммирование результатов вычисления потоков.

Строка 37 - 59: Вычисление интеграла в потоках.

Результат выполнения программы

Нижняя граница Верхняя граница Шаг интегрирования

Нижняя граница	Верхняя граница	Шаг	Результат
1.0	1.0E8	0.5	0.091114186867...

Добавить
Удалить
Рассчитать

run:
Vremya vipolneniya: 2663 ms

Рисунок 3 — Результат

Вывод

Изучена работа с потоками. В некоторых задачах лучше не использовать потоки для оптимизации кода.